

All programs were tested with the `wsc` CLI. Valid programs are run with `./wsc <file>`; the `--dump-ast` flag prints the parsed AST to stdout.

## 1 Valid Programs

The table below summarises which language features each example exercises.

| Feature                                         | Ex. 1 | Ex. 2 | Ex. 3 |
|-------------------------------------------------|-------|-------|-------|
| <code>task</code> declaration (structured type) | ✓     | ✓     | ✓     |
| Function definition ( <code>fun</code> )        | ✓     | ✓     | ✓     |
| Function call                                   | ✓     | ✓     | ✓     |
| Task field access ( <code>t.field</code> )      | ✓     | ✓     | ✓     |
| <code>if</code> statement                       | ✓     | ✓     | ✓     |
| <code>else</code> branch                        | ✓     |       |       |
| <code>while</code> loop                         |       |       | ✓     |
| pipeline + <code>stage</code>                   |       |       | ✓     |
| <code>on_failure</code>                         |       |       | ✓     |
| <code>run</code> statement                      | ✓     | ✓     | ✓     |

### 1.1 Example 1 — Conditional Dispatch (01\_basic.ws)

**What it exercises.** Two `task` declarations (the structured type), a function that receives a `task` and reads its fields to compute a cost, variable declarations, and an `if/else` that runs the cheaper task. This is the primary example for the `else` branch.

```

1  // Conditional dispatch: choose the cheaper task based on computed cost
2  task fast {
3      timeout: 20
4      retries: 0
5      parallel: true
6  }
7
8  task thorough {
9      timeout: 90
10     retries: 3
11     parallel: false
12 }
13
14 fun total_cost(t: task) → int {
15     return t.timeout * (t.retries + 1)
16 }
17
18 var fast_cost: int = total_cost(fast)
19 var thorough_cost: int = total_cost(thorough)
20
21 if fast_cost < thorough_cost {
22     run fast
23 } else {
24     run thorough
25 }
```

Listing 1: 01\_basic.ws — conditional dispatch based on task cost

Parsed successfully `./wsc examples/valid/01_basic.ws` exits with code 0, no errors produced.

### 1.2 Example 2 — Task Inspection Functions (02\_functions.ws)

**What it exercises.** Multiple function definitions including one with two `int` parameters and one that takes a `task` parameter and accesses its `timeout` field. Demonstrates function calls used as initializers in `var` declarations and an `if` that gates a `run` statement.

```

1 // User-defined functions and task field access
2 task deploy { timeout: 120 retries: 1 parallel: false }
3
4 fun max_time(t: int, r: int) → int {
5     return t * (r + 1)
6 }
7
8 fun is_long(t: task) → bool {
9     return t.timeout > 60
10 }
11
12 var m: int = max_time(120, 1)
13 var slow: bool = is_long(deploy)
14
15 if slow {
16     run deploy
17 }

```

Listing 2: 02.functions.ws — function definitions and task field access

Parsed successfully — `./wsc examples/valid/02_functions.ws` exits with code 0, no errors produced.

### 1.3 Example 3 — Full CI/CD Pipeline (03\_pipeline.ws)

**What it exercises.** All domain-specific constructs: `pipeline`, `stage`, `on_failure`, and `run`. The `while` loop (the only loop control structure) increments an attempt counter. A function call is used as the condition of the final `if`. Together with Examples 1 and 2 this closes coverage of the full language.

```

1 // Full pipeline with on_failure, functions, while loop
2 task build { timeout: 30 retries: 0 parallel: false }
3 task test  { timeout: 60 retries: 2 parallel: true  }
4 task deploy { timeout: 120 retries: 1 parallel: false }
5
6 pipeline release {
7     stage build
8     stage test
9     stage deploy
10    on_failure { run build }
11 }
12
13 fun should_run(limit: int) → bool {
14     return limit > 0
15 }
16
17 var attempt: int = 0
18 var limit: int = 3
19
20 while attempt < limit {
21     attempt = attempt + 1
22 }
23
24 if should_run(limit) {
25     run release
26 }

```

Listing 3: 03\_pipeline.ws — pipeline with on.failure, while loop, and conditional run

Parsed successfully — `./wsc examples/valid/03_pipeline.ws` exits with code 0, no errors produced.

## 2 Malformed Programs

Each subsection gives the source file, the exact error string printed to `stderr` by `wsc`, and a brief explanation of which grammar rule the input violates.

### 2.1 Error 1 — Missing Closing Brace (01\_missing\_brace.ws)

```
1 task build { timeout: 30 retries: 0 parallel: false
```

Listing 4: 01\_missing\_brace.ws

```
line 2: expected RBRACE, got ""
```

**Explanation.** The grammar rule for `<task_decl>` requires a closing `}` after the three field declarations. The source ends immediately after the `false` token, so the parser reaches end-of-file while still expecting `RBRACE`. The “got `""`” indicates the EOF token, which has an empty lexeme.

### 2.2 Error 2 — Wrong Task Field Order (02\_bad\_field\_order.ws)

```
1 task build { retries: 0 timeout: 30 parallel: false }
```

Listing 5: 02\_bad\_field\_order.ws

```
line 1: expected KW_TIMEOUT, got "retries"
```

**Explanation.** The grammar mandates a fixed field order: `timeout`, then `retries`, then `parallel`. Swapping `retries` and `timeout` violates this ordering; the parser immediately reports that it expected the keyword `timeout` but found `retries` instead.

### 2.3 Error 3 — Empty Pipeline (03\_empty\_pipeline.ws)

```
1 pipeline ci { }
```

Listing 6: 03\_empty\_pipeline.ws

```
line 1: pipeline must have at least one stage
```

**Explanation.** The grammar rule `<pipeline_body> ::= <stage_stmt> { <stage_stmt> } [ <on_failure> ]` requires at least one `stage` statement. An empty body violates this constraint. The parser checks `len(stages) == 0` after the stage-collection loop and emits a semantic parse error before the closing `}` is consumed.

### 2.4 Error 4 — Missing Arrow in Function Signature (04\_missing\_arrow.ws)

```
1 fun double(x: int) int { return x }
```

Listing 7: 04\_missing\_arrow.ws

```
line 1: expected ARROW, got "int"
```

**Explanation.** The grammar requires the `->` separator between the parameter list and the return type: `<fun_decl> ::= "fun" IDENT "(" ... ")" "->" <type> <block>`. Omitting `->` causes the parser to see the return-type keyword `int` where it expects the `ARROW` token.

## 2.5 Error 5 — Integer Literal for Boolean Field (05\_bad\_parallel.ws)

```
1 task build { timeout: 30 retries: 0 parallel: 1 }
```

Listing 8: 05\_bad\_parallel.ws

```
line 1: expected BOOL_LIT, got "1"
```

**Explanation.** The grammar specifies "parallel" ":" BOOL\_LIT; only the tokens `true` and `false` satisfy BOOL\_LIT. The integer literal `1` is lexed as INT\_LIT, which does not match the expected token type. This demonstrates that WorkflowScript does not perform implicit numeric-to-boolean coercion, even at parse time.